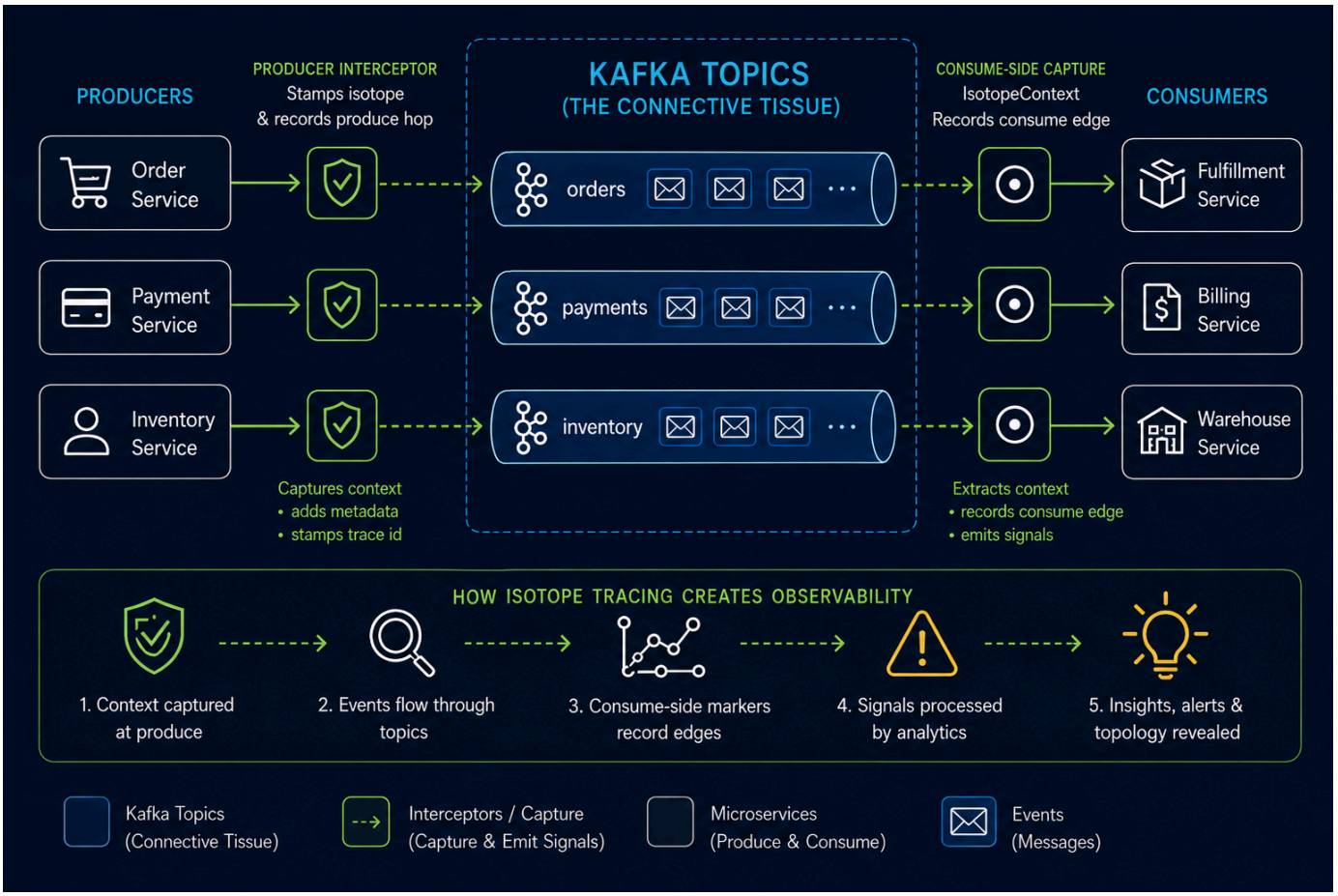
Confluent Kafka Isotope

Table of Contents

- Purpose
- 1.0 How an Isotope Traverses an Event Pipeline
  - 1.1 Anatomy of the Isotope Tracing Artifact
- 2.0 Architecture
- 3.0 Getting Started
  - 3.1 Integration Tests with Confluent Platform on minikube
  - 3.2 Seven Scalar Headers Flink SQL Reports with Apache Flink on minikube
  - 3.3 Seven Scalar Headers Flink SQL Reports with Confluent Cloud for Apache Flink
    - 3.3.1 Why `latency_percentiles` is a `ProcessTableFunction` (PTF)
    - 3.3.2 [OPTIONAL] Eighth Report — AI Root-Cause Analysis (RCA)
  - 3.4 [OPTIONAL] Prometheus Metrics Reporting with Grafana Visualization
  - 3.5 Apache Flink Provenance
    - 3.5.1 [ALWAYS ON] 1:1 Collector Provenance: Flink as a Hop
    - 3.5.2 [OPTIONAL] Fan-in Provenance: Windowed Merges
    - 3.5.3 [OPTIONAL] State-Level Provenance: Upsert and CDC Sources
    - 3.5.4 [OPTIONAL] Iceberg Lakehouse: Trace History and Traversable Provenance
  - 3.6 [OPTIONAL] Reports Health Monitor
- Resources
  - Companion Articles
  - Companion Presentation
  - Confluent Platform
  - Confluent Cloud
  - Miscellaneous

Purpose

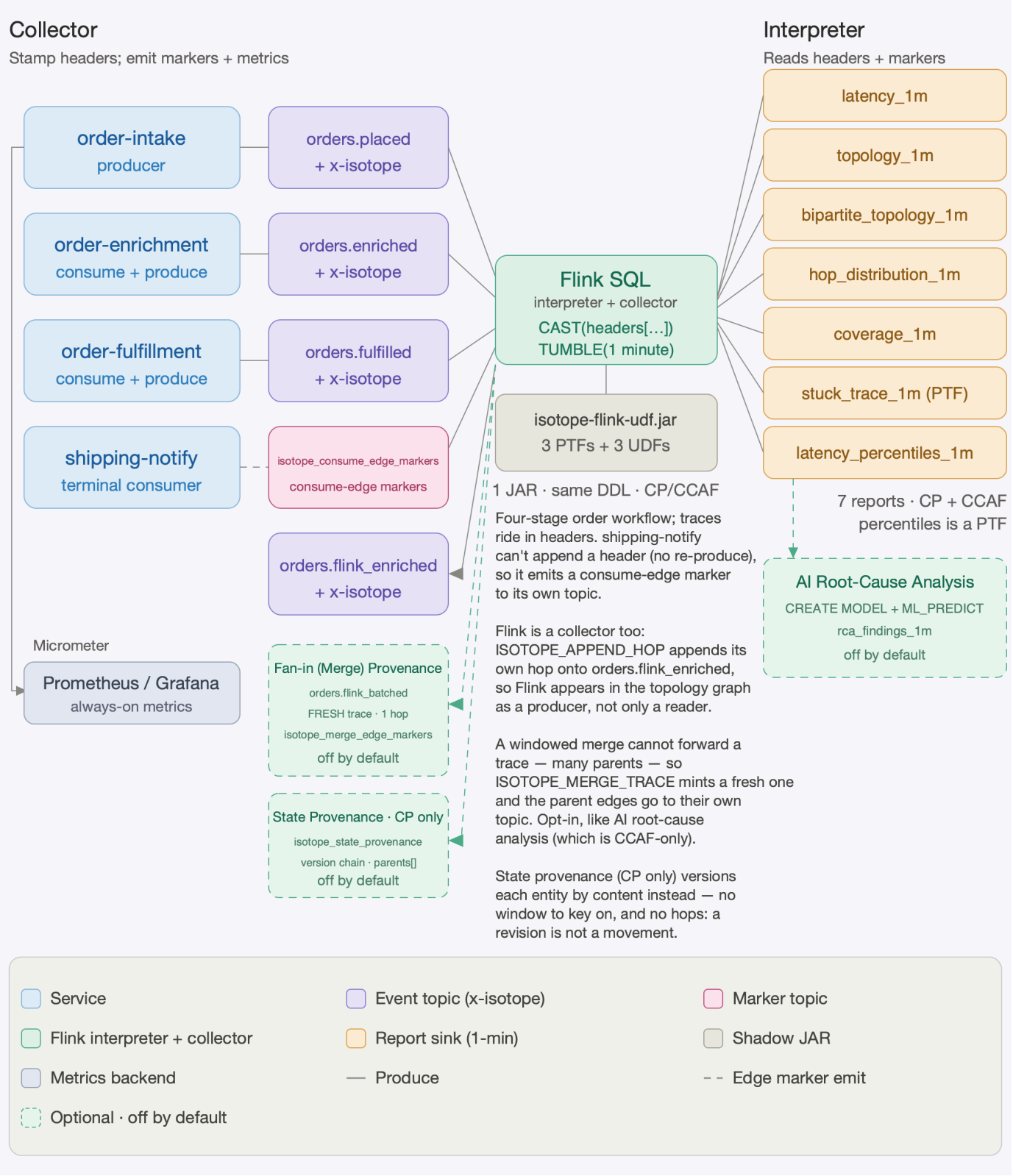
`confluent-kafka-isotope` is a reference implementation of an ***e-commerce order pipeline that uses Kafka Interceptors to capture end-to-end event-tracing data, with Prometheus and Apache Flink analyzing and reporting that data in batch and near real time***, as visualized below.



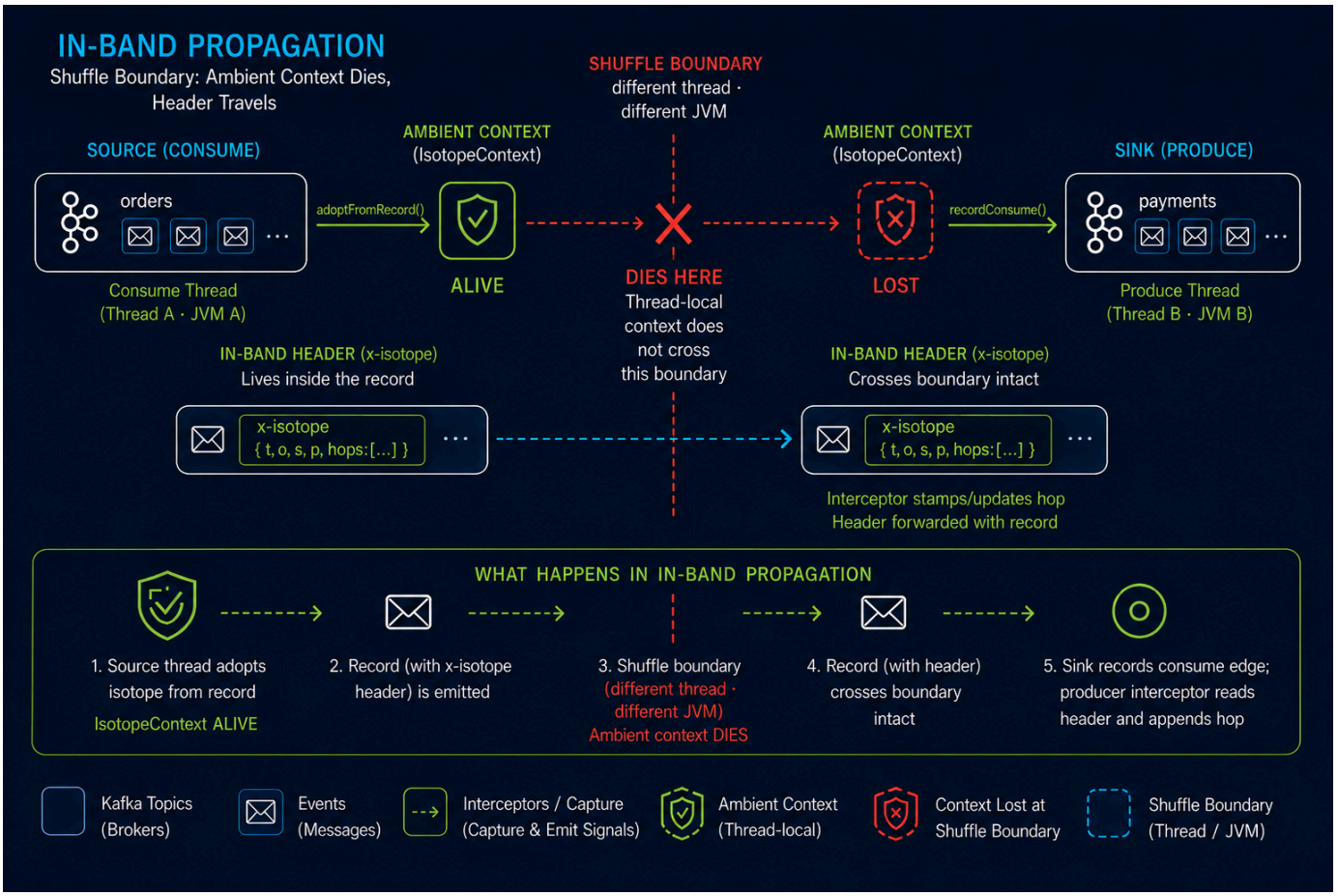
Much like isotopes used to trace molecules through a biochemical pathway, each event carries lightweight metadata that allows it to be followed as it travels through Kafka topics and distributed microservices.

Kafka topics become the **connective tissue between services**, while **Kafka Interceptors quietly transform** the event pipeline itself into an **observable distributed system**.

This project demonstrates how **Kafka Interceptors become collectors** — inserting the isotope into record headers in place and, at terminal consumers, emitting consume-edge markers — while **Flink SQL serves as the interpreter**, reading those headers directly to produce seven 1-minute reports from a single JAR on both Confluent Platform and Confluent Cloud for Apache Flink (CCAF). Optionally, the producer interceptor can **emit Micrometer metrics to Prometheus as an always-on aggregate layer** alongside the per-trace Flink reports. (As depicted in the diagram below.)



Flink is now a collector too. A second propagation model lets a Flink statement stamp its own hop, so Flink appears in the topology graph as a producer rather than only as a reader. The interceptor propagates *out-of-band* — the in-flight isotope lives in an `IsotopeContext` `ThreadLocal` that `onSend()` reads on the same thread — which is correct in the services, where one thread owns a whole consume→produce hop. That cannot work in Flink: a shuffle resumes a record on a different thread in a different JVM, and Flink SQL has no user-visible thread to attach to at all. So Flink propagates *in-band*, carrying the isotope in the record's own headers via a writable `headers` metadata column and the `ISOTOPE_APPEND_HOP` scalar UDF. Two models, one wire format — every header the UDF writes is byte-identical to the interceptor's, so the reports cannot tell the two collectors apart. See docs/flink-collector.md.



A third collector records state, not messages. The first two both record *message* lineage, and an isotope is an itinerary — one identity and its ordered hops — which is a truthful derivation record only while every hop is 1:1. A row in an upsert table has no itinerary: its current value is the fold of every change that ever touched it, a DAG over versions rather than a path over messages. Asking whether a `−U`/`+U` pair is one hop or two has no good answer, because the question is wrong. So the third collector stamps **no hops at all**. `STATE_PROVENANCE` publishes one record per emitted state, identified by its *content and event time* rather than by a window, carrying the versions it was derived from inline — which is what lets a lineage stream describing an *updating* table stay append-only end to end. Deletes are included, as first-class versions. Opt-in on both runtimes, and the version IDs are identical across them; see docs/state-provenance.md.

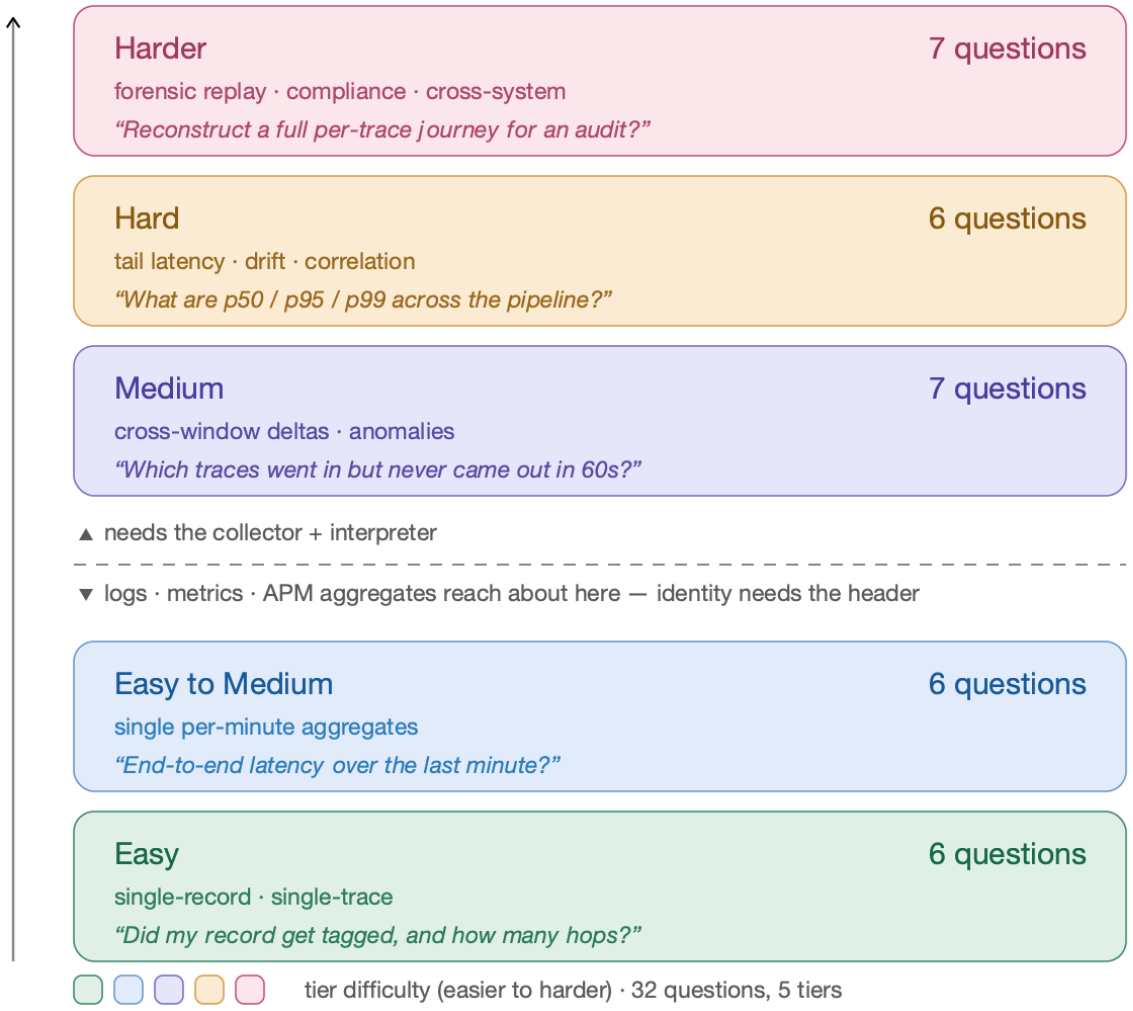
That makes three collectors, distinguished by what they can truthfully record:

Collector	Records	Propagation	Availability
`IsotopeProducerInterceptor`	message lineage — one hop per `send()`	out-of-band (`ThreadLocal`)	always on
`ISOTOPE_APPEND_HOP`	message lineage — Flink's own hop, 1:1 statements only	in-band (record headers)	always on; opt-in fan-in variant for windowed merges mints a fresh trace rather than forwarding one (§3.5.2)
`STATE_PROVENANCE`	state lineage — a content-addressed version chain per entity	neither: a parallel record keyed by version	opt-in, CP + CCAF (§3.5.3)

The first two share one wire format, so no report can tell them apart. The third deliberately does not participate in it — which is the point: overloading the hop list to mean "revision" would fabricate movement that never happened.

With this approach, developers gain **end-to-end observability** into the flow of events through the Kafka-based microservices architecture, enabling both **real-time monitoring** and **post-hoc analysis** of event traces.

This end-to-end observability of the isotope tracing pipeline creates a **ladder of insights**, allowing developers to trace each event's journey, identify bottlenecks, and improve the performance and reliability of the entire system. The ladder is organized into **five tiers that answer 32 questions**, as visualized below. All 32 are enumerated, tier by tier, in docs/metrics.md §6.



- ▶ Harder (7 questions) — forensic replay, compliance, cross-system joins
- ▶ Hard (6 questions) — tail behavior, drift detection, correlated analysis
- ▶ Medium (7 questions) — cross-window deltas, anomalies, multi-report joins
- ▶ Easy to Medium (6 questions) — single per-minute aggregates
- ▶ Easy (6 questions) — single-record, single-trace

1.0 How an Isotope Traverses an Event Pipeline

An **Isotope** is a ***lightweight tracing artifact attached to Kafka record headers***. Like a biochemical isotope used to trace molecules through a metabolic pathway, it allows the journey of a record through an event-driven architecture to be observed and analyzed.

This project models a Kafka pipeline as a **bipartite graph**: **services occupy one vertex set**, **topics the other**, and **every produce and consume operation forms an edge between them**. The resulting graph provides a unified view of the complete event topology, from producers to intermediate processing services to terminal consumers.

A `ProducerInterceptor` stamps the isotope onto each record and appends one hop for every `send()` operation, capturing the **produce edges**. Consumers call `IsotopeContext.recordConsume()` to emit a lightweight marker representing the corresponding **consume edges**.

For services that consume and then produce, Kafka Isotope supports **two complementary propagation models** for carrying the trace identity forward:

- **Out-of-band propagation** — `IsotopeContext.adoptFromRecord()` places the inbound isotope into thread-local context, where the subsequent `send()` retrieves it. This model is designed for conventional Kafka applications where the consume → produce hop remains within the same execution context.
- **In-band propagation** — the isotope remains attached to the record as it moves through the processing graph. Because propagation does not depend on thread-local state, the trace identity can survive thread, task, shuffle, network, and JVM boundaries such as those introduced by Apache Flink.

Both models produce the same Kafka isotope headers and preserve the same trace identity across hops; they differ only in **how that identity is propagated between consume and produce**: out-of-band through thread-local context, or in-band with the record itself.

Apache Flink combines the isotope headers and consume-edge markers to reconstruct the complete **service → topic → service** graph and produce seven reports:

- **End-to-end latency**
- **Latency percentiles**
- **Produce-side topology**
- **Full bipartite topology**
- **Hop distribution**
- **Per-topic coverage** — a trace-loss funnel signal
- **Stuck-trace detection**

The same implementation runs on both **Confluent Platform (self-managed)** with **Confluent Manager for Apache Flink (CMF)** and **Confluent Cloud for Apache Flink (CCAF)**.

For more in-depth discussion of the **Isotope Tracing Design**, including the two propagation models, see docs/design.md.

1.1 Anatomy of the Isotope Tracing Artifact

The isotope is a **JSON object** that travels in the `x-isotope` Kafka record header, accompanied by **seven scalar headers** that Flink SQL reads directly. The JSON carries the trace identity, origin metadata, and full ordered hop history, while the scalar headers provide a flattened view of the most-recent-hop data for easier SQL access.

- **Header `x-isotope`** (JSON bytes) carries the full trace state and hop history, forwarded by every hop:
  - `t` — 16-byte **UUIDv7** trace ID (RFC 9562): 48-bit millisecond timestamp in the high bits + 74 random bits. It remains stable for the life of the trace, and lexicographic byte order matches creation order — sorting trace IDs therefore produces chronological creation order. The random bits come from `ThreadLocalRandom`, not `SecureRandom`: a trace ID is a public observability identifier carried in Kafka headers, so the requirement is collision avoidance rather than unpredictability, without imposing `SecureRandom` overhead on every produced record.
  - `o` — origin timestamp (ms), identical to the timestamp embedded in the UUIDv7 trace ID; retained as a separate field for typed access from Flink SQL without decoding the UUID bytes.
  - `s` — origin service name, stamped once and forwarded unchanged for the life of the trace.
  - `p` — origin pipeline name (for example, `orders` vs. `location`); like `s`, stamped once at the origin and forwarded unchanged on every hop, allowing reports to slice traces by logical pipeline.
  - `h` — ordered list of hops, each represented as `{s: service, t: topic, m: tsMs}`.
  - `x` — `true` if the hop list exceeded `MAX_HOPS = 32` and the oldest hop was evicted.
- **Seven scalar headers** (UTF-8 strings) carry the most-recent-hop view, allowing Flink SQL to read them directly via `CAST(headers['x-isotope-…'] AS STRING)` without parsing the JSON hop array or requiring a UDF on either CCAF or CP Flink. See scripts/flink/README.md for the complete scalar-header table.

▶ Sample Isotope Tracing Artifact

2.0 Architecture

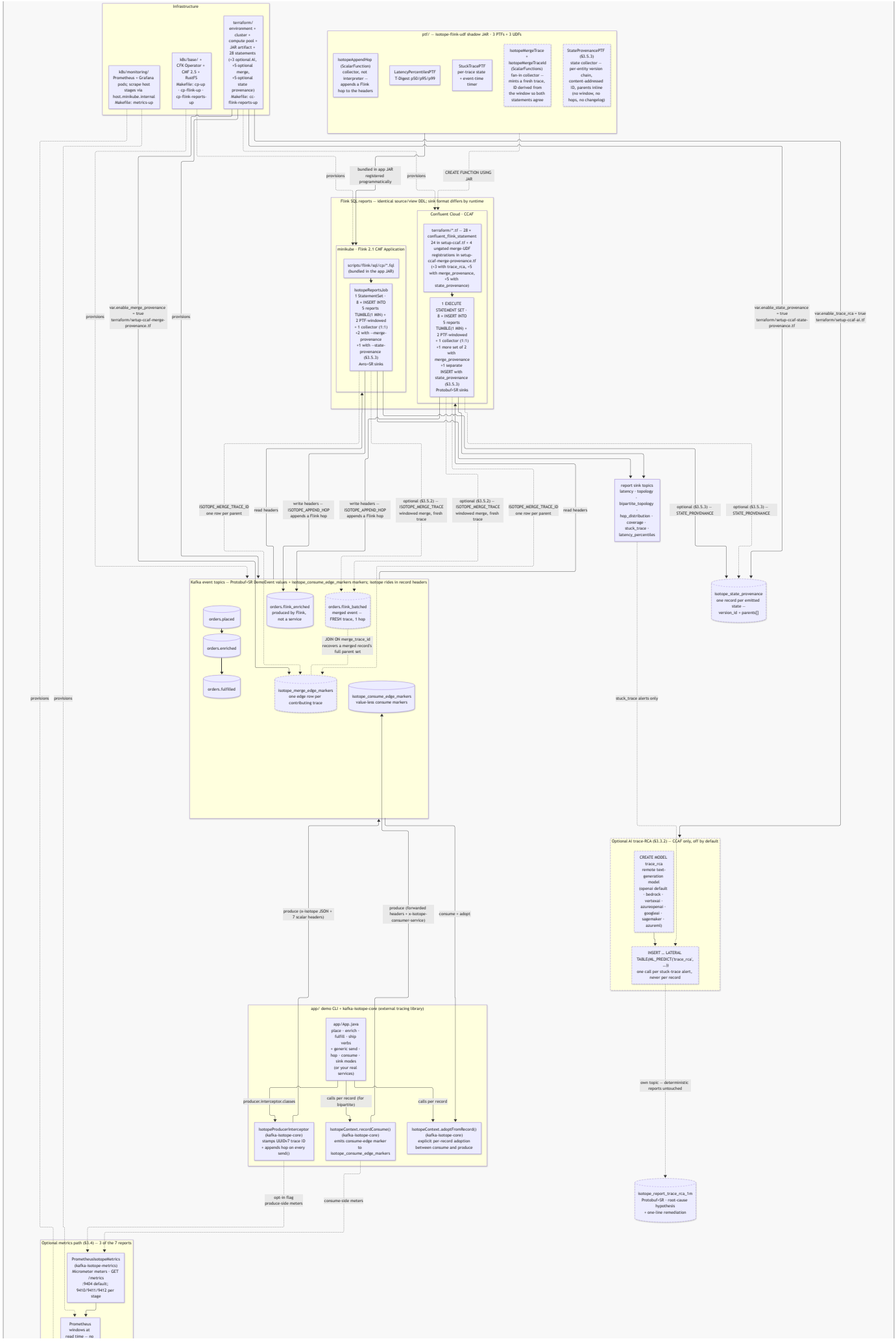
A bird's-eye view of the moving parts. The demo CLI in `app/` consumes the external tracing library (`ai.signalroom:kafka-isotope-core`), which registers a Kafka `ProducerInterceptor` that stamps an isotope into record headers on every `send()`. Consume-then-produce services propagate the inbound trace by explicitly calling `IsotopeContext.adoptFromRecord(record)`. Business events then flow through a three-topic Kafka pipeline, where Flink SQL reads the isotope metadata and emits one-minute aggregate reports.

Both runtimes run the same seven logical reports off the same source and view definitions, though each runtime keeps its own copy of the SQL. **Confluent Platform (CP) + Flink** on minikube executes the `.fql` files under `scripts/flink/sql/cp/` as a single Flink 2.1 Confluent Manager for Apache Flink (CMF) Application (`IsotopeReportsJob`), while **Confluent Cloud for Apache Flink (CCAF)** applies the same logical SQL as inline `confluent_flink_statement` Terraform resources under `terraform/`. The shadow JAR from `ptf/`—which powers two of the seven reports plus the collector UDF—runs unchanged on both runtimes: bundled into the CP application JAR and uploaded as a Flink artifact on CCAF.

Alongside the seven reports, both runtimes run one **collector** INSERT that forwards `orders.placed` to `orders.flink_enriched`, appending a Flink hop on the way. Each side runs all eight as a **single job**: CP through `IsotopeReportsJob`'s `StatementSet`, CCAF through `EXECUTE STATEMENT SET BEGIN ... END`. That matters for cost as much as symmetry — every separate CCAF statement carries its own 1-CFU floor, so eight statements meant eight floors.

Alongside that—**additive, opt-in, and disabled by default**—the interceptor can also emit **Micrometer** metrics for **Prometheus**, with **Grafana** providing visualization (§3.4 Prometheus Metrics Reporting with Grafana Visualization). This path produces the three **stateless** reports (`latency_1m`, `topology_1m`, and `hop_distribution_1m`) without requiring a stream processor. The remaining four reports continue to run in Flink because they depend on per-`trace_id` state or absence-of-event analysis, which falls outside Prometheus's query model.

(Kafka is drawn once below for brevity; each runtime provisions its own Kafka cluster.)


Infrastructure
terraform/ environment + cluster + compute pool + JAR artifact + 28 statements (+3 optional AI, +5 optional merge, +5 optional state provenance) Makefile: cc-flink-reports-up
k8s/base/ + CFK Operator + CMF 2.5 + RustFS Makefile: cp-up · cp-flink-up · cp-flink-reports-up
k8s/monitoring/ Prometheus + Grafana pods; scrape host stages via host.minikube.internal Makefile: metrics-up
ptf/ — isotope-flink-udf shadow JAR · 3 PTFs + 3 UDFs
IsotopeAppendHop (ScalarFunction) collector, not interpreter — appends a Flink hop to the headers
LatencyPercentilesPTF T-Digest p50/p95/p99
StuckTracePTF per-trace state + event-time timer
IsotopeMergeTrace + IsotopeMergeTraceId (ScalarFunctions) fan-in collector — mints a fresh trace_id derived from the window so both statements agree
StateProvenancePTF ($3.5.3) state collector — per-entity version chain, content-addressed ID, parents inline (no window, no hops, no changelog)
provisions
bundled in app JAR registered programmatically
provisions
CREATE FUNCTION USING JAR
Flink SQL reports — identical source/view DDL; sink format differs by runtime
Confluent Cloud · CCAF
terraform/*.tf — 28 × confluent_flink_statement 24 in setup-ccaf.tf + 4 ungated merge-UDF registrations in setup-ccaf-merge-provenance.tf (+3 with trace_rca, +5 with merge_provenance, +5 with state_provenance)
minikube · Flink 2.1 CMF Application
scripts/flink/sql/cp/*.fql (bundled in the app JAR)
IsotopeReportsJob 1 StatementSet · 8 × INSERT INTO 5 reports TUMBLE(1 MIN) + 2 PTF-windowed + 1 collector (1:1) +2 with --merge-provenance +1 with --state-provenance ($3.5.3) Avro+SR sinks
1 EXECUTE STATEMENT SET · 8 × INSERT INTO 5 reports TUMBLE(1 MIN) + 2 PTF-windowed + 1 collector (1:1) +1 more set of 2 with merge_provenance +1 separate INSERT with state_provenance ($3.5.3) Protobuf-SR sinks
provisions
var.enable_merge_provenance = true terraform/setup-ccaf-merge-provenance.tf
provisions
var.enable_state_provenance = true terraform/setup-ccaf-state-provenance.tf
var.enable_trace_rca = true terraform/setup-ccaf-ai.tf
ISOTOPE_MERGE_TRACE_ID one row per parent
read headers
write headers — ISOTOPE_APPEND_HOP appends a Flink hop
write headers — ISOTOPE_APPEND_HOP appends a Flink hop
optional ($3.5.2) — ISOTOPE_MERGE_TRACE windowed merge, fresh trace
optional ($3.5.2) — ISOTOPE_MERGE_TRACE windowed merge, fresh trace
ISOTOPE_MERGE_TRACE_ID one row per parent
read headers
report sink topics latency · topology · bipartite_topology · hop_distribution · coverage · stuck_trace · latency_percentiles
optional ($3.5.3) — STATE_PROVENANCE
optional ($3.5.3) — STATE_PROVENANCE
Kafka event topics — Protobuf-SR Demo
Event values + isotope_consume_edge_markers markers; isotope rides in record headers
orders.placed
orders.enriched
orders.fulfilled
orders.flink_enriched produced by Flink, not a service
orders.flink_batched merged event — FRESH trace, 1 hop
JOIN ON merge_trace_id recovers a merged record's full parent set
isotope_merge_edge_markers one edge row per contributing trace
isotope_consume_edge_markers value-less consume markers
isotope_state_provenance one record per emitted state — version_id + parents[]
provisions
provisions
stuck_trace alerts only
Optional AI trace-RCA ($3.3.2) — CCAF only, off by default
CREATE MODEL trace_rca remote text-generation model (openai default · bedrock · vertexai · azureopenai · googleai · sagemaker · azureml)
INSERT … LATERAL TABLE(ML_PREDICT('trace_rca', …)) one call per stuck-trace alert, never per record
produce (x-isotope JSON + 7 scalar headers)
produce (forwarded headers + x-isotope-consumer-service)
consume + adopt
app/ demo CLI + kafka-isotope-core (external tracing library)
app/App.java place · enrich · fulfill · ship verbs + generic send · hop · consume · sink modes (or your real services)
producer.interceptor.classes
calls per record (for bipartite)
calls per record
IsotopeProducerInterceptor (kafka-isotope-core) stamps UUIDv7 trace ID + appends hop on every send()
IsotopeContext.recordConsume() (kafka-isotope-core) emits consume-edge marker to isotope_consume_edge_markers
IsotopeContext..adoptFromRecord() (kafka-isotope-core) explicit per-record adoption between consume and produce
own topic — deterministic reports untouched
opt-in flag produce-side meters
consume-side meters
isotope_report_trace_rca_1m Protobuf-SR · root-cause hypothesis + one-line remediation
Optional metrics path ($3.4) — 3 of the 7 reports
PrometheusIsotopeMetrics (kafka-isotope-metrics) Micrometer meters · GET /metrics :9404 default; 9410/9411/9412 per stage
Prometheus windows at read time — no



▶ See Repo Layout

3.0 Getting Started

First, clone the repo to your local machine using the GitHub CLI:

```
gh repo clone j3-signalroom/confluent-kafka-isotope
```

Change to the repo directory:

```
cd /path/to/confluent-kafka-isotope
```

Then decide how you want to run the repo:

3.1 Integration Tests with Confluent Platform on minikube

```
make install-prereqs     # VM driver (vfkit | kvm2/qemu), kubectl, minikube, helm, gettext,
gradle, openjdk17 — on Linux: sudo make install-prereqs
make check-prereqs       # verify they're on PATH
```

Default minikube sizing: `MINIKUBE_CPUS=6`, `MINIKUBE_MEM=20480` (MiB), and `MINIKUBE_DISK=50g`. Override via env, `make` args, or `local.mk`. Sizing is fixed when the cluster is created, so run `make minikube-delete` before changing it.

minikube runs as a **VM with containerd** as the container runtime, so Docker isn't required. The VM driver is picked per OS and can be overridden with `MINIKUBE_DRIVER=`:

Host	Default driver	Alternative
macOS (Apple silicon or Intel)	`vfkit` (Apple Virtualization framework)	—
Linux x86_64	`kvm2` (KVM via libvirt)	`qemu`
Linux arm64	`qemu`	—

On Linux, both drivers need hardware virtualization (`/dev/kvm`; on a cloud VM that means nested virtualization). `sudo make install-prereqs` adds your user to the `kvm` group (plus `libvirt` for kvm2), so log out and back in afterwards. Run `make minikube-driver-check` to verify the driver setup. **Node can't pull images but the host can?** Set `MINIKUBE_HTTP_PROXY` to an HTTP proxy on the host in a git-ignored `local.mk` (see `local.mk.example`), and see KNOWN_ISSUES.md §2.0.

Bring up the local Confluent Platform stack and port-forward Kafka + SR:

```
make minikube-start                  # one-time
make cp-up                           # CFK Operator + Kafka/SR/Connect/ksqlDB/C3 (~5 min)
make kafka-pf-up                     # localhost:30092 → Kafka, localhost:8081 → Schema
Registry
```

Then run the suite:

```
./gradlew :app:integrationTest                                          # every IT below
./gradlew :app:integrationTest --tests '*ProducerInterceptorIT'         # just one
```

Override the endpoints if needed:

```
./gradlew :app:integrationTest \
    -PkafkaBootstrap=localhost:30092 \
    -PschemaRegistryUrl=http://localhost:8081
```

Tear down forwards when done:

```
make kafka-pf-down
```

The integration tests cover:

Test	What it verifies
`BrokerSmokeIT`	AdminClient can create/list/delete a topic via the NodePort port-forward
`ProducerInterceptorIT`	A consumer sees the `x-isotope` JSON header + all 7 scalar reporting headers with the expected origin/hop values, and the Protobuf round-trip preserves `DemoEvent.source` / `payload`
`ThreeStageHopPropagationIT`	`order-intake-service → topic-AB → order-enrichment-service → topic-BC → order-fulfillment-service` produces a stable trace ID, 2-hop trail in send order, and correct scalar headers (origin = `order-intake-service`, this = `order-enrichment-service`, hop count = 2) at the terminal; consume-then-produce hops use `IsotopeContext.adoptFromRecord` to carry the trace forward
`BipartiteTopologyIT`	The 4-stage `order-intake-service → topic-AB → order-enrichment-service → topic-BC → order-fulfillment-service → topic-CD → shipping-notification-service` chain emits exactly three consume-edge markers to a per-test markers topic — one per consume edge. Every marker carries the trace ID, forwarded `x-isotope-*` scalars describing the upstream producer, and the new `x-isotope-consumer-service` naming the downstream consumer. Asserts the `(consumer_service, consumed_topic)` set is exactly the three pairs of stages 2-4

3.2 Seven Scalar Headers Flink SQL Reports with Apache Flink on minikube

> **Caveat:** minikube is a single-node cluster. It is not a production-like environment, but it is sufficient for local development and testing. The Confluent Platform (CP) + Flink stack runs in minikube, and you can port-forward Kafka and Schema Registry to your local machine.

To ***run***, ***test***, and ***debug*** Apache Flink like a production engineer, this project provides a full Confluent Platform + Flink stack running locally on minikube — no cloud required.

You get an environment on your machine, with all the components you'd expect in a real deployment:

- **Confluent Platform** (KRaft mode) via Confluent for Kubernetes (CFK)
- **Apache Flink 2.1.2** via the Confluent Flink Kubernetes Operator 1.140.1
- **Confluent Manager for Apache Flink (CMF) 2.4.0** for Flink environment management

To run this project, you'll need **macOS (with Homebrew)** or **Linux (with apt-get)**. The full stack — **minikube + Confluent Platform + Flink + CMF** — is resource-intensive and designed to mirror an adequate development environment. Therefore,

the following defaults for minikube are recommended: `MINIKUBE_CPUS=6`, `MINIKUBE_MEM=20480` (MiB), and `MINIKUBE_DISK=50g`.

> These settings ensure stable performance across all components. You can tune them as needed, but lower resource levels may cause pod restarts or degraded performance.

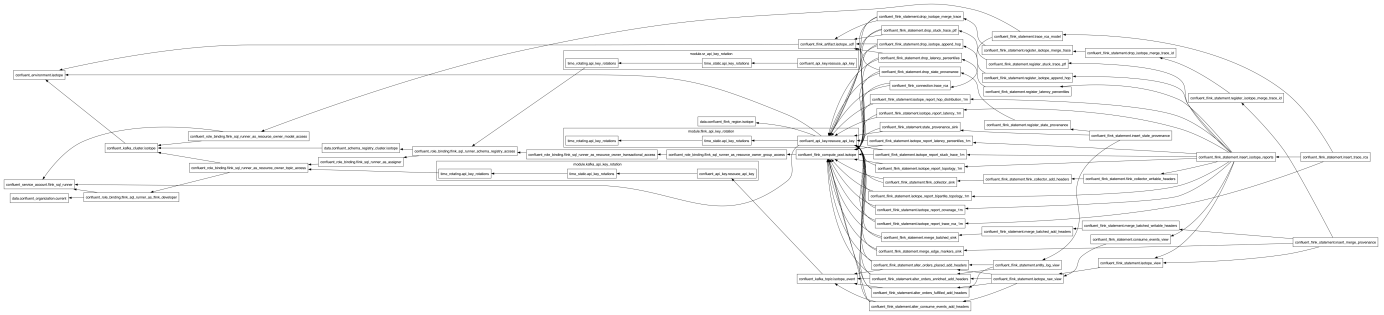
Seven reports — five pure Flink SQL plus two JAR-backed PTFs — and the collector INSERT run as a **single Flink 2.1 CMF Application** (`IsotopeReportsJob`, one StatementSet, and eight sinks) submitted through CMF and executed by the Confluent Flink Kubernetes Operator. One StatementSet means one failure domain: a fault in any INSERT stops them all, so a missing sink topic takes the reports down with it (which is why `deploy-cmf-flink-reports.sh` pre-creates every sink — unlike CCAF, OSS Flink's Kafka connector declares a table over a topic that must already exist rather than creating it). No raw session cluster is involved; `k8s/base/flink-cluster-deployment.yaml` (`make flink-deploy` / `make flink-sql`) is a separate, optional path for ad-hoc SQL. Confluent Cloud runs the same seven reports from its own copy of the SQL, inlined in Terraform — see §3.3 Confluent Cloud for Apache Flink for that path; this section is the local-minikube one.

The full bring-up sequence — cluster → Flink → reports → traffic → teardown — is consolidated in docs/runbook-local.md. The short version: `make cp-flink-up` then `make cp-flink-reports-up`, then drive traffic across **multiple** 1-minute windows (a single burst sits in one open window forever — the watermark has to cross `window_end` for a tumbling window to emit) and wait ~90s after the last record.

Report sink topics ride **Avro+SR** (`avro-confluent`, auto-registered on first write) so Control Center renders them natively — a deliberate *format-by-domain* split: app events are **Protobuf+SR** (`DemoEvent`), Flink aggregates are **Avro+SR** (cp-flink ships no SR-integrated Protobuf format), and the consume-edge marker topic `isotope_consume_edge_markers` is **null-value / headers-only**. Not a defect — a clean split by domain.

3.3 Seven Scalar Headers Flink SQL Reports with Confluent Cloud for Apache Flink

The Confluent Cloud for Apache Flink (CCAF) parallel of §3.2 CP + Apache Flink on minikube, driven by Terraform under terraform/. The Terraform graph below depicts the resources deployed in Confluent Cloud.



`make cc-flink-reports-up CONFLUENT_API_KEY=... CONFLUENT_API_SECRET=...` provisions a fresh Confluent Cloud environment containing:

- Kafka cluster
- Compute pool
- Two rotating service-account API key pairs (Kafka + Schema Registry)
- The uploaded PTF JAR
- 24 `confluent_flink_statement` resources (6 `ALTER TABLE` + 3 `VIEW` + 8 sink `CREATE TABLE` + 3 `CREATE FUNCTION`, plus 1 `EXECUTE STATEMENT SET` holding all 8 `INSERT INTO`s and 3 transient `DROP FUNCTION`)

The full provision → deploy → traffic → teardown sequence is consolidated in docs/runbook-cloud.md. `make cc-flink-reports-up` (~6–8 min first run; idempotent re-applies), drive traffic with `scripts/cc-app-run.sh place|enrich|fulfill|ship` across **multiple** 1-minute windows (a single burst sits in one open window forever, and you wait ~90s after the last record), then `make cc-flink-reports-down` to `terraform destroy` the whole environment.

Format-by-runtime (not-by-domain). CP's reports land on **Avro+SR** (`'value.format' = 'avro-confluent'` in scripts/flink/sql/cp/05_report_sinks.fql). CCAF's reports land on **Protobuf+SR** (`'value.format' = 'proto-registry'` in each sink's `WITH` clause in terraform/setup-ccaf.tf). The two runtimes' SQL is otherwise unshared: CP's lives hardcoded in scripts/flink/sql/cp/, CCAF's lives inline as `confluent_flink_statement` resources in terraform/setup-ccaf.tf.

3.3.1 Why `latency_percentiles` is a `ProcessTableFunction` (PTF)

Since CCAF does not support User-Defined AGGregate functions (UDAGG), I implemented the percentiles report as a `ProcessTableFunction` to keep it portable across runtimes. `LATENCY_PERCENTILES` (class `LatencyPercentilesPTF`) performs its own 1-minute tumbling-window aggregation over a T-Digest sketch using per-window state and event-time timers. **A PTF avoids that UDAGG restriction, so it registers and runs on both runtimes, just like `STUCK_TRACE_PTF`. Both runtimes therefore run the same seven reports:** `latency` (avg/min/max), `topology` (produce-side), `bipartite_topology` (full service↔topic↔service graph), `hop_distribution`, `coverage`, `stuck_trace`, and `latency_percentiles` (p50/p95/p99).

3.3.2 [OPTIONAL] Eighth Report — AI Root-Cause Analysis (RCA)

In addition to the seven deterministic reports, an **eighth, AI-generated report** turns each stuck-trace *alert* into a natural-language root-cause hypothesis and a one-line remediation. This **CCAF-only capability** is configured by `terraform/setup-ccaf-ai.tf` and can be enabled **at deploy time** by running:

```
make cc-flink-reports-up  CONFLUENT_API_KEY=... CONFLUENT_API_SECRET=...
ENABLE_TRACE_RCA=true RCA_MODEL_API_KEY=... RCA_MODEL_PROVIDER=... RCA_MODEL_VERSION=...
RCA_MODEL_ENDPOINT=...
```

The deployment provisions the standard CCAF environment used by the seven deterministic reports:

- Kafka cluster
- Compute pool
- Two rotating service-account API key pairs (Kafka + Schema Registry)
- The uploaded PTF JAR
- 24 `confluent_flink_statement` resources (6 `ALTER TABLE` + 3 `VIEW` + 8 sink `CREATE TABLE` + 3 `CREATE FUNCTION`, plus 1 `EXECUTE STATEMENT SET` holding all 8 `INSERT INTO`s and 3 transient `DROP FUNCTION`)

When `ENABLE_TRACE_RCA=true`, the deployment **additionally provisions the AI RCA resources**:

- `CREATE MODEL trace_rca` — registers a remote text-generation model.
- A `proto-registry` sink table `isotope_report_trace_rca_1m`.
- An `INSERT … SELECT … LATERAL TABLE(ML_PREDICT('trace_rca', …))` that calls the model **once per alert** (reading the low-volume 1-minute stuck-trace report topic, never the source stream) and writes the hypothesis to its own topic — it never overwrites the deterministic reports.

> The default provider is OpenAI (`gpt-4o`). Claude is supported via **AWS Bedrock** (`rca_model_provider = "bedrock"` with AWS credentials). Other supported providers: `vertexai`, `azureopenai`, `googleai`, `sagemaker`, `azureml`.

3.4 [OPTIONAL] Prometheus Metrics Reporting with Grafana Visualization

Three of the seven reports — `latency_1m`, `topology_1m`, and `hop_distribution_1m` — are pure **stateless scalar aggregations** over bounded-cardinality dimensions (`service` / `topic` / `hop_count`, never `trace_id`). These don't require a stream processor: the producer interceptor already has every value in scope on every `send()`, so it emits them as **Micrometer** meters (PrometheusIsotopeMetrics.java). **Prometheus** performs the windowed aggregations at query time, with **Grafana** providing the visualization layer.

The other four reports — `latency_percentiles`, `coverage`, `bipartite_topology`, and `stuck_trace` — require per-`trace_id` state or absence-of-event detection that Prometheus cannot express, so they **remain in Flink**. This path is **additive and opt-in**: a **3-Micrometer / 4-Flink split**.

> **Full details** — including the meter/PromQL reference, the produce- and consume-side signals, the two deliberate gaps (`distinct_traces`, windowed `min`), and why latency percentiles remain implemented as a PTF — are in **docs/metrics.md**. The one-command Prometheus + Grafana showcase has its own runbook: **k8s/monitoring/README.md** (`make metrics-up`).

3.5 Apache Flink Provenance

In this repo, ***Apache Flink provenance is the record of the data's origin and how it was derived*** *— the hops it traversed, the records that were aggregated into it, and the updates and deletes that replaced its earlier versions*.

	1:1 Collector	[OPTIONAL] Fan-in Provenance	[OPTIONAL] State-Level Provenance
Answers	*"Where has this message been?"*	*"Which inputs produced this merged output?"*	*"How did this entity's state evolve?"*
Relationship	A path: one parent per hop	Many parents to one output	A chain of versions per entity
Carried	Inside the record's headers	Alongside it, on its own topic	Alongside it, parents listed inline
Mechanism	`ISOTOPE_APPEND_HOP` scalar UDF	`ISOTOPE_MERGE_TRACE` + `ISOTOPE_MERGE_TRACE_ID` UDFs, two INSERTs	`STATE_PROVENANCE` PTF, one statement
Output	`orders.flink_enriched`	`orders.flink_batched` + `isotope_merge_edge_markers`	`isotope_state_provenance`
Runtimes	CP + CCAF	CP + CCAF	CP + CCAF
Default	Always on	Off (`ENABLE_MERGE_PROVENANCE=true`)	Off (`ENABLE_STATE_PROVENANCE=true`)

3.5.1 [ALWAYS ON] 1:1 Collector Provenance: Flink as a Hop

- **Why the header approach:** the services' `ThreadLocal` approach can't survive a Flink shuffle, so Flink writes the hop into the record's own `headers` column.
- **Why the planner keeps it:** the header value is part of the query's output, so no optimizer rewrite can drop it.
- **Result:** Flink shows up as an ordinary hop, `hop_count = 2`, with the original trace continued.
- **Scope:** only for 1:1 statements. An aggregate never passes a parent's trace forward. A `SUM` over 1,000 records has 1,000 parents, so naming one would be made-up provenance (flink-collector.md §3.1).

3.5.2 [OPTIONAL] Fan-in Provenance: Windowed Merges

- **Fresh trace:** the merged record starts a new trace, and a separate topic records one edge per contributing trace per window. That edge carries a `contributing_records` count for weight.
- **Derived ID:** two statements must agree on `merge_trace_id` without talking to each other. The ID is a UUIDv7 computed from the window bounds and group key, so replays produce the same value.
- **Two checks that must hold:** the edge count for a merged record equals its `distinct_traces`, and `SUM(contributing_records)` equals its `event_count`.
- **Limits:**
  - It only works over tumbling windows, since it needs a fixed input set to hash.
  - The two merge SQL statements must be kept in sync by hand.
  - The edge topic adds a material fraction of that stage's write volume.
- **Ancestry:** depth is exactly 1 today, because merge output can't feed back into a merge.

Both runtimes use the same switch, off by default:

```
# CP runtime
make cp-flink-reports-up ENABLE_MERGE_PROVENANCE=true

# CC runtime
make cc-flink-reports-up ENABLE_MERGE_PROVENANCE=true \
    CONFLUENT_API_KEY=$CONFLUENT_API_KEY CONFLUENT_API_SECRET=$CONFLUENT_API_SECRET
```

> For more information, refer to **docs/flink-collector.md §2.4**.

3.5.3 [OPTIONAL] State-Level Provenance: Upsert and CDC Sources

- **Different model:** it's for data that changes in place. Each emitted state gets a content-addressed `version_id` (event time + SHA-256 of source, key and content), and its `parents` are carried in the same record.
- **Why it avoids the merge collector's problems:**
  - No window is needed.
  - One operator writes each version together with its parents, so the two can't disagree.
  - It reads the underlying log in append mode and rebuilds state itself, so no operator sees an update.
  - Revisions become new, immutable versions appended to the topic.
- **Deletes count:** tombstones get provenance too.
- **Same IDs on both runtimes:** both hash the raw value bytes — CP's `'value.format' = 'raw'` `value` column, and the `val` column CCAF's Topic Catalog imports for the order topics. Before hashing, `StateVersion` strips the Schema Registry header (the magic byte and the 4-byte schema ID), because the schema ID is assigned by whichever registry the producer used. The same record therefore gets the same `version_id` on CP and CCAF.
- **Queryable, not traversable, in Flink:** Flink SQL has no recursive CTEs, so walking a version chain end to end needs the lakehouse (§3.5.4).

To enable state-level provenance, use the following `make` commands:

```
# CP runtime
make cp-flink-reports-up ENABLE_STATE_PROVENANCE=true

# CC runtime
make cc-flink-reports-up ENABLE_STATE_PROVENANCE=true \
    CONFLUENT_API_KEY=$CONFLUENT_API_KEY CONFLUENT_API_SECRET=$CONFLUENT_API_SECRET

# both collectors at once — they answer different questions
make cp-flink-reports-up ENABLE_MERGE_PROVENANCE=true ENABLE_STATE_PROVENANCE=true
```

> For more information, refer to **docs/state-provenance.md**.

3.5.4 [OPTIONAL] Iceberg Lakehouse: Trace History and Traversable Provenance

The reports keep 1-minute aggregates, and the raw isotope lives only as long as Kafka retention. The lakehouse keeps the rows, in Apache Iceberg tables on the RustFS instance CMF already uses:

- **`lakehouse.isotope.hops`:** one row per produced hop, so any trace can be pulled back at any age, and a new question is a SQL query rather than a new Flink job.
- **`lakehouse.isotope.state_provenance`:** the state-provenance stream (with `ENABLE_STATE_PROVENANCE=true`). Flink SQL has no recursive CTEs, so version chains are *queryable, not traversable* there; in the lakehouse, `WITH RECURSIVE` walks every chain end to end.
- **Separate failure domain:** the lakehouse is a second CMF Application (`IsotopeLakehouseJob`, same JAR), so a catalog or storage fault can't stop the seven reports.
- **CP only:** the lakehouse runs as a CMF Application, so it is CP only. State provenance itself runs on both runtimes.

To enable the lakehouse, use the following `make` commands, then query it from the host with DuckDB (`brew install duckdb`):

```
make cp-flink-reports-up ENABLE_LAKEHOUSE=true ENABLE_STATE_PROVENANCE=true

scripts/lakehouse-query.sh            # every entity's version chain, walked recursively
scripts/lakehouse-query.sh latency    # per-topic p50/p95/p99 over the lakehouse's whole
history
scripts/lakehouse-query.sh shell      # interactive DuckDB with the lakehouse attached
```

> For more information, refer to **docs/lakehouse.md**.

3.6 [OPTIONAL] Reports Health Monitor

Stuck Trace runs inside the reports job, so when that job stops, its alerts stop too, and the silence looks like nothing being stuck. The health monitor runs separately (a second CMF Application on CP, its own statement on CCAF, same JAR) and watches the reports job through its topics: once traffic on `orders.placed` has moved past a one-minute window, `isotope_report_topology_1m` should publish that window. If it doesn't within 2 minutes, the monitor writes a `STALLED` row to `isotope_reports_health`, then a `RECOVERED` row when the report appears.

```
# CP runtime
make cp-flink-reports-up ENABLE_HEALTH_MONITOR=true

# CC runtime
make cc-flink-reports-up ENABLE_HEALTH_MONITOR=true CONFLUENT_API_KEY=...
CONFLUENT_API_SECRET=...
```

- **When it alerts:** expect `STALLED` about 2 to 3 minutes after the reports stop. That covers a dead reports job and a frozen reports watermark, because both stop windows from closing.
- **What `RECOVERED` means:** the reports job has published the overdue window, not that it has caught up. A restarted job replays its backlog, so it can still be behind.
- **Traffic has to keep flowing:** the monitor runs on event time, so if `orders.placed` goes quiet it can't alert, even if the reports job is dead too.
- **Shared dependencies:** it publishes to Kafka, so it can't report a Kafka outage. On CP it has its own pods but shares CMF and the Flink operator with the reports job. On CCAF it shares the compute pool and Confluent Cloud.
- **A stopped stage on CCAF:** CCAF's idleness detection can grow to 5 minutes, so a stopped stage can hold the reports back for longer than the 2-minute grace period and raise `STALLED`. The reports really have stopped publishing, but the cause is the stage, not the job.

Resources

Companion Articles

- Medium Article: Kafka's quiet observability superpower — Kafka Interceptors
- Medium Article: Kafka's quiet observability superpower — Kafka Interceptors with assistance from AI

Companion Presentation

- Current 2026 San Francisco (Confluent) Presentation: Kafka's quiet observability superpower — Kafka Interceptors

Confluent Platform

- Confluent Platform Overview
- Deploy and Manage Confluent Platform Using Confluent for Kubernetes
- Stream Processing with Confluent Platform for Apache Flink
- minikube

Confluent Cloud

- Confluent Cloud Overview
- Stream Processing Concepts in Confluent Cloud for Apache Flink

Miscellaneous

- What is Makefile and make? How do we use it?
- Apache Flink Kickstarter II